

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Proyecto:

Invernadero Inteligente IoT con Motor ARM64 de Decisión y Análisis Histórico

PONDERACIÓN: 60 pts

 **Tiempo estimado: 70 hrs**

Índice

1. MARCO FORMATIVO.....	4
1.1. Valor.....	4
1.2. Competencia(s).....	4
1.3. Habilidad(es) blandas a formar.....	4
2. Resultado del Aprendizaje.....	4
2.1. Objetivo SMART.....	4
3. Resumen Ejecutivo.....	5
4. Enunciado del Proyecto.....	6
4.1 Descripción del problema o necesidad a resolver.....	6
4.2 Alcance del proyecto.....	6
Requisito previo obligatorio: Fase 1 funcional.....	6
Elementos mínimos requeridos de Fase 1.....	6
4.3 Alcance obligatorio de Fase 2.....	7
4.4 Componente A: Motor ARM64 en vivo.....	8
Descripción general.....	8
Flujo obligatorio del motor en vivo.....	8
Formato de entrada sugerido.....	9
4.5 Historial interno de lecturas.....	9
4.6 Biblioteca común utils.s.....	10
4.7 Indicadores mínimos para toma de decisiones.....	11
Promedio reciente.....	11
Tendencia acumulada.....	11
Amplitud reciente.....	12
4.8 Aplicación de indicadores por sensor.....	12
4.9 Decisión principal.....	12
Valores de referencia para la toma de decisiones.....	13
4.10 Acciones posibles.....	14
4.11 Formato de salida del motor en vivo.....	15
Opción A: salida en una sola línea.....	15
Opción B: salida en múltiples líneas.....	15
4.12 Manejo de errores.....	15
4.13 Componente B: Analizador histórico ARM64.....	16
4.14 Entrada del analizador.....	16
4.15 Columnas válidas.....	16
4.16 Validaciones obligatorias.....	17
4.17 Actualización de módulos ARM64 existentes.....	17
4.18 Salida del analizador.....	18
4.19 Nuevos cálculos ARM64 de Fase 2.....	18
4.19.1 Rutina 1: RMSE respecto a un valor ideal.....	18

4.19.2 Rutina 2: Regresión lineal simple.....	19
4.19.3 Rutina 3: Predicción futura por regresión.....	20
4.19.4 Rutina 4: Integral del error por regla del trapecio.....	21
4.19.5 Rutina 5: Derivada suavizada por regresión local.....	21
Restricciones obligatorias.....	23
4.20 Integración entre Python y ARM64.....	23
Responsabilidades de Python.....	23
Responsabilidades de ARM64.....	24
4.22 Registro de resultados en MongoDB.....	24
4.23 Visualización en Grafana y dashboard web.....	25
Paneles mínimos sugeridos en Grafana.....	25
Dashboard web.....	25
4.24 Login y despliegue público.....	26
4.24 Recursos y herramientas a utilizar.....	26
4.25 Entregables.....	27
5. Material de apoyo.....	28
6. Metodología.....	28
Pasos a seguir.....	28
7. Rúbrica de Calificación.....	30
7.1 Requisitos para optar a la calificación.....	30
7.2 Detalle de la Calificación.....	32
7.3 Comentarios Generales.....	35

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad y trabajo colaborativo	Se aplica mediante la continuidad del proyecto desarrollado en la Fase 1, integrando nuevos módulos ARM64, procesamiento histórico de datos, despliegue web, visualización con Grafana y toma de decisiones automatizada. Cada integrante deberá desarrollar, explicar y defender un módulo ARM64 propio, además de participar en la integración grupal del sistema completo.

1.2. Competencia(s)

Con la elaboración de esta segunda fase, el estudiante adquirirá la siguiente competencia:

Integra sistemas embebidos, programación en bajo nivel, análisis de datos históricos y despliegue web mediante la ampliación de un invernadero inteligente IoT basado en Raspberry Pi, Python y ARM64, incorporando un motor de decisión en vivo, procesamiento de archivos grandes, análisis gráfico de sensores, visualización con Grafana y acceso remoto mediante una aplicación web desplegada.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Trabajo en equipo.
- Organización y distribución técnica de responsabilidades.
- Resolución de problemas de integración hardware/software.
- Defensa técnica individual de módulos ARM64.
- Comunicación de resultados mediante dashboard y Grafana.
- Documentación técnica de sistemas embebidos.
- Integración de componentes desarrollados por distintos miembros del equipo.
- Análisis crítico de decisiones automáticas tomadas por software de bajo nivel.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Ampliar el Invernadero Inteligente IoT de Fase 1 mediante un	Implementando al menos un módulo ARM64 por integrante, un motor en	Usando Raspberry Pi, Python, ARM64, MongoDB Atlas,	Para relacionar programación de bajo nivel, sistemas embebidos,	Durante el período establecido para la Fase 2 del proyecto.

motor de decisión en ARM64, análisis histórico de archivos grandes, visualización en Grafana y despliegue web público.	vivo por stdin/stdout, análisis de archivos con cantidad variable de filas, selección de rango inicio-fin, dashboard con login, despliegue en la nube y visualización en Grafana.	MQTT, Grafana, dashboard web y servicios de despliegue como Render, Vercel o Amazon S3.	análisis temporal de sensores, toma de decisiones automatizada y visualización remota de datos.	
--	---	---	---	--

3. Resumen Ejecutivo

La segunda fase del proyecto toma como base obligatoria el Invernadero Inteligente IoT desarrollado en la Fase 1. Para optar a calificación en esta fase, el sistema anterior deberá estar funcional, incluyendo maqueta física, sensores, actuadores, lectura de datos, control de riego, ventilación, iluminación, alarma, MQTT, MongoDB Atlas, dashboard web, LCD, botones físicos y documentación técnica.

En esta fase, el sistema deberá evolucionar hacia una arquitectura donde ARM64 tenga participación directa en la toma de decisiones y en el análisis histórico de datos. Python continuará siendo responsable de leer sensores, controlar GPIO, ejecutar actuadores, publicar datos, almacenar información y comunicarse con servicios externos. Sin embargo, ARM64 deberá encargarse de calcular indicadores, generar decisiones principales y analizar datos históricos.

El proyecto se divide en dos componentes principales. El primer componente consiste en un motor ARM64 en vivo, ejecutado como proceso desde Python, que recibe lecturas actuales, almacena un historial reciente, calcula indicadores de comportamiento y responde indicando qué acción debe ejecutar Python. El segundo componente consiste en un analizador histórico ARM64 capaz de procesar archivos de lecturas, seleccionar un rango válido de líneas y analizar una columna de sensor como una serie temporal.

Además, se agregará visualización avanzada con Grafana, registro de resultados ARM64 en MongoDB, actualización del dashboard web, login y despliegue público.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

En la Fase 1, el invernadero inteligente permitió construir una base funcional de monitoreo y control mediante Raspberry Pi, sensores, actuadores, comunicación MQTT, almacenamiento en MongoDB Atlas y visualización mediante un dashboard web. El sistema fue capaz de leer variables ambientales y del suelo, ejecutar acciones automáticas y registrar información relevante del funcionamiento del invernadero.

Sin embargo, un sistema embebido más avanzado no debe depender únicamente de reglas simples programadas en Python. En un invernadero real, las decisiones de riego, ventilación, iluminación o alarma pueden depender del comportamiento reciente de los sensores, de la tendencia de las variables, de la diferencia acumulada entre lecturas y del análisis histórico de los datos.

Por esta razón, en la Fase 2 se incorpora procesamiento ARM64 para realizar análisis de datos y apoyar la toma de decisiones. Python continuará siendo el coordinador general del sistema, pero la decisión automática principal deberá ser calculada por ARM64.

La finalidad de esta fase es que el sistema evolucione de un invernadero automatizado hacia una arquitectura híbrida donde Python coordina el sistema y ARM64 realiza procesamiento de bajo nivel para generar indicadores, decisiones y resultados históricos.

4.2 Alcance del proyecto

Requisito previo obligatorio: Fase 1 funcional

Para optar a calificación en Fase 2, el grupo deberá demostrar que el sistema desarrollado en la Fase 1 se encuentra funcional.

No se calificará la Fase 2 si el sistema base no cumple con los elementos principales de la fase anterior, ya que esta segunda fase depende directamente de la integración física y lógica previamente construida.

Elementos mínimos requeridos de Fase 1

Elemento	Requisito
Maqueta física	Debe representar un invernadero funcional con dos áreas de cultivo y centro de control.
Raspberry Pi	Debe usarse como unidad principal del sistema.

Sensores	Deben estar integrados sensores ambientales, sensores de suelo, sensor de luz y sensor de gas.
Actuadores	Deben funcionar riego, ventilación, luces, buzzer y LEDs de estado.
Python	Debe leer sensores, controlar GPIO, ejecutar actuadores y coordinar el sistema.
MQTT	Debe enviar lecturas y recibir comandos.
MongoDB Atlas	Debe registrar lecturas, eventos, comandos, estados y logs.
Dashboard web	Debe mostrar monitoreo y permitir control remoto.
LCD y botones físicos	Deben mostrar información y permitir control local.
Documentación	Debe explicar arquitectura, conexión, ejecución y funcionamiento.

4.3 Alcance obligatorio de Fase 2

La Fase 2 deberá cumplir obligatoriamente con los siguientes elementos:

1. Mantener funcional todo lo desarrollado en Fase 1.
2. Implementar un motor ARM64 en vivo que reciba datos desde Python.
3. Mantener un historial reciente de lecturas en memoria.
4. Calcular indicadores de comportamiento reciente en ARM64.
5. Generar una decisión principal desde ARM64.
6. Ejecutar desde Python la acción indicada por ARM64.
7. Evitar que Python calcule la decisión automática principal.
8. Implementar análisis histórico ARM64 sobre archivos de lecturas con cantidad variable de filas.
9. Permitir seleccionar una línea inicial y una línea final.
10. Validar que el rango solicitado exista dentro del archivo.
11. Analizar una sola columna seleccionada.
12. Considerar el orden de las lecturas como referencia temporal.
13. Actualizar los cálculos ARM64 desarrollados previamente para trabajar con rangos variables.
14. Implementar nuevos cálculos ARM64 de Fase 2.
15. Implementar al menos un módulo ARM64 por integrante.
16. Usar una biblioteca común llamada utils.s para operaciones reutilizables.
17. Integrar Grafana para visualización avanzada de datos, resultados e indicadores.
18. Desplegar la aplicación web en un servicio accesible públicamente.
19. Incorporar login en la aplicación web.
20. Registrar en MongoDB los resultados generados por ARM64.
21. Documentar y defender individualmente cada módulo ARM64.

Para considerar que la Fase 2 cumple con el alcance obligatorio, no será suficiente con ejecutar programas ARM64 de forma aislada. El sistema deberá demostrar una integración completa entre sensores reales, Python, ARM64, MongoDB, dashboard, Grafana y actuadores físicos.

4.4 Componente A: Motor ARM64 en vivo

Descripción general

El motor ARM64 en vivo será un programa ejecutado y controlado desde Python. Python deberá enviar lecturas actuales del invernadero al proceso ARM64 mediante entrada estándar stdin. ARM64 procesará los datos recibidos, actualizará su historial interno, calculará indicadores de comportamiento reciente y devolverá una decisión estructurada mediante salida estándar stdout.

Python deberá interpretar la respuesta generada por ARM64 y ejecutar la acción correspondiente sobre los actuadores físicos del invernadero. Posteriormente, deberá registrar en MongoDB la lectura original, los indicadores calculados, la decisión ARM64, la acción ejecutada y el resultado de la operación.

Flujo obligatorio del motor en vivo

Paso	Responsable	Descripción
1	Python	Lee sensores reales desde Raspberry Pi.
2	Python	Construye una línea de datos con las lecturas actuales.
3	Python	Envía la línea al proceso ARM64 mediante stdin.
4	ARM64	Recibe y parsea los valores enviados.
5	ARM64	Valida que la entrada tenga el formato esperado.
6	ARM64	Inserta la lectura actual en su historial interno.
7	ARM64	Calcula indicadores sobre el historial actualizado.
8	ARM64	Determina una acción automática.
9	ARM64	Imprime por stdout una respuesta estructurada.
10	Python	Lee e interpreta la respuesta de ARM64.
11	Python	Ejecuta la acción indicada sobre los actuadores.

12	Python	Registra lectura, decisión, indicadores y acción en MongoDB.
13	Dashboard y Grafana	Muestran lectura, decisión, indicadores y resultados.

Formato de entrada sugerido

Cada lectura enviada desde Python hacia ARM64 deberá contener los valores principales del invernadero en una sola línea.

Formato sugerido:

TEMP,HUM_AIRE,SOIL1,SOIL2,LUZ,GAS,MOD0

Ejemplo:

31,68,34,41,280,160,0

Campo	Descripción
TEMP	Temperatura ambiental del invernadero.
HUM_AIRE	Humedad ambiental.
SOIL1	Humedad del suelo del área de cultivo 1.
SOIL2	Humedad del suelo del área de cultivo 2.
LUZ	Valor del sensor de iluminación.
GAS	Valor del sensor de gas.
MOD0	Estado del sistema: automático o manual. Se usará 0 para automático y 1 para manual.

4.5 Historial interno de lecturas

El motor ARM64 deberá mantener un historial interno de las últimas lecturas recibidas. Este historial podrá implementarse mediante arreglos de tamaño fijo. El tamaño mínimo recomendado será de 5 lecturas.

Cada sensor podrá almacenarse en un arreglo independiente.

Ejemplo conceptual:

Sensor	Arreglo sugerido
Temperatura	temp_buffer
Humedad ambiental	hum_buffer
Humedad suelo 1	soil1_buffer
Humedad suelo 2	soil2_buffer
Luz	luz_buffer
Gas	gas_buffer

Cada vez que Python envíe una nueva lectura, ARM64 deberá insertarla en los arreglos correspondientes. Luego de insertar la lectura actual, ARM64 deberá calcular los indicadores usando el historial actualizado.

Los indicadores no se calculan únicamente sobre el último resultado ni únicamente sobre la última lectura. Se calculan sobre el conjunto reciente de valores almacenados en los arreglos.

El motor podrá organizarse en uno o varios archivos ARM64. El uso de múltiples archivos será válido cuando facilite la separación de lectura, almacenamiento, indicadores, decisión y salida.

4.6 Biblioteca común `utils.s`

Se deberá reutilizar la biblioteca común `utils.s` desarrollada en la Fase 1, realizando las adaptaciones necesarias para que pueda trabajar con archivos de lecturas de tamaño variable.

En la fase anterior, `utils.s` estaba orientada a leer y procesar un conjunto fijo de 30 datos. Para esta segunda fase, deberá modificarse para permitir el procesamiento de rangos seleccionados por el usuario, definidos mediante una línea inicial, una línea final y una columna específica.

La biblioteca `utils.s` deberá conservar las funciones reutilizables ya implementadas, como:

- lectura del archivo de datos;
- separación de registros y columnas;
- conversión ASCII a entero;
- conversión entero a ASCII;
- almacenamiento temporal de valores;
- recorrido de datos;
- escritura de resultados;
- validaciones básicas.

Además, deberá adaptarse para:

- procesar únicamente las líneas comprendidas dentro del rango indicado;
- contar dinámicamente la cantidad de valores procesados;
- validar que la columna seleccionada exista;
- permitir que los módulos ARM64 trabajen con una cantidad variable de datos;
- compartir funciones comunes entre los módulos individuales.

4.7 Indicadores mínimos para toma de decisiones

El motor ARM64 deberá calcular indicadores sobre el historial reciente. se define un conjunto estándar de dos indicadores principales que pueden aplicarse a todos los sensores:

1. Promedio reciente.
2. Tendencia acumulada.

Cada sensor podrá usar estos dos indicadores.

Promedio reciente

El promedio reciente se calcula sobre los valores almacenados en el arreglo del sensor.

Fórmula:

$$\text{PROMEDIO} = \text{suma}(X_i) / N$$

Donde:

- X_i representa cada lectura almacenada en el arreglo.
- N representa la cantidad de lecturas disponibles en el historial.

Tendencia acumulada

La tendencia acumulada mide si los valores recientes aumentan, disminuyen o se mantienen estables.

Fórmula:

$$\text{DIF}_i = X_i - X_{(i-1)}$$

$$\text{DIF_ACUM} = \text{suma}(\text{DIF}_i)$$

Clasificación:

Condición	Tendencia
$\text{DIF_ACUM} > 0$	Tendencia ascendente

DIF_ACUM < 0	Tendencia descendente
DIF_ACUM = 0	Tendencia estable

Amplitud reciente

La amplitud reciente mide la diferencia entre el valor máximo y el valor mínimo dentro del historial.

Fórmula:

AMPLITUD = MAXIMO - MINIMO

Este indicador podrá usarse para detectar variaciones fuertes en sensores como gas, luz o temperatura.

4.8 Aplicación de indicadores por sensor

Sensor	Indicadores mínimos	Uso esperado
TEMP	Promedio reciente y tendencia acumulada	Apoyar decisiones de ventilación.
HUM_AIRE	Promedio reciente y amplitud	Observar estabilidad del ambiente.
SOIL1	Promedio reciente y tendencia acumulada	Apoyar decisiones de riego en área 1.
SOIL2	Promedio reciente y tendencia acumulada	Apoyar decisiones de riego en área 2.
LUZ	Promedio reciente y tendencia acumulada	Apoyar decisiones de iluminación.
GAS	Promedio reciente y amplitud	Apoyar decisiones de alarma o ventilación.

El motor no deberá aplicar únicamente comparaciones directas sobre la lectura actual. La decisión deberá apoyarse al menos en el promedio reciente y la tendencia acumulada, o en el promedio reciente y la amplitud reciente.

4.9 Decisión principal

El motor ARM64 emitirá una única acción principal por cada lectura recibida.

La decisión deberá basarse en los indicadores calculados previamente. La decisión deberá seguir el siguiente orden de prioridad:

Prioridad	Condición evaluada	Acción sugerida
1	Promedio reciente de GAS elevado o amplitud reciente de GAS elevada	ALARM_ON
2	Promedio reciente de SOIL1 bajo y tendencia acumulada descendente	RIEGO_1_ON
3	Promedio reciente de SOIL2 bajo y tendencia acumulada descendente	RIEGO_2_ON
4	Promedio reciente de LUZ bajo y tendencia acumulada descendente	LIGHT_ON
5	Promedio reciente de TEMP alto y tendencia acumulada ascendente	FAN_ON
6	Estado general sin condición crítica	LED_GREEN
7	Sin acción física requerida	NO_ACTION

Los valores de referencia para clasificar bajo, alto o elevado deberán estar documentados por el grupo. Dichos valores no deberán ser calculados por Python para tomar la decisión principal; Python solo enviará los datos y ejecutará la acción recibida.

Valores de referencia para la toma de decisiones

Para que ARM64 pueda clasificar un promedio como bajo, alto o elevado, el grupo deberá definir valores de referencia para cada sensor.

Estos valores deberán estar documentados y deberán usarse dentro del motor ARM64 para evaluar las condiciones de decisión.

Los valores de referencia podrán definirse según las pruebas realizadas con los sensores, el comportamiento esperado del invernadero o los valores utilizados durante la Fase 1.

Tabla de referencia sugerida:

Variable	Valor de referencia	Uso
GAS_ALTO	Definido por el grupo	Determinar si el promedio reciente de gas es elevado.

GAS_AMP_ALTA	Definido por el grupo	Determinar si la variación reciente del gas es elevada.
SOIL_BAJO	Definido por el grupo	Determinar si la humedad del suelo es baja.
LUZ_BAJA	Definido por el grupo	Determinar si el nivel de luz es bajo.
TEMP_ALTA	Definido por el grupo	Determinar si la temperatura es alta.

El motor ARM64 deberá comparar los indicadores calculados contra estos valores de referencia.

Python no deberá calcular si una variable es baja, alta o elevada. Python únicamente enviará las lecturas al motor ARM64 y ejecutará la acción indicada.

4.10 Acciones posibles

El motor ARM64 podrá devolver las siguientes acciones:

Acción	Descripción
RIEGO_1_ON	Activar riego en área de cultivo 1.
RIEGO_2_ON	Activar riego en área de cultivo 2.
FAN_ON	Activar ventilación.
LIGHT_ON	Activar iluminación artificial.
ALARM_ON	Activar alarma por gas o condición crítica.
LED_GREEN	Indicar estado normal.
LED_YELLOW	Indicar advertencia.
LED_RED	Indicar riesgo alto o crítico.
NO_ACTION	No ejecutar acción física.

Python deberá validar que la acción recibida sea una acción permitida antes de ejecutarla en GPIO.

4.11 Formato de salida del motor en vivo

ARM64 deberá devolver una respuesta estructurada que Python pueda interpretar de forma directa.

Se permitirá cualquiera de los siguientes formatos:

Opción A: salida en una sola línea

ACTION=RIEGO_1_ON;TARGET=SOIL1;RISK=HIGH;REASON=SOIL_LOW_AND_DESCENDING;VALUE=34;INDICATOR=-6

Opción B: salida en múltiples líneas

ACTION=RIEGO_1_ON
TARGET=SOIL1
RISK=HIGH
REASON=SOIL_LOW_AND_DESCENDING
VALUE=34
INDICATOR=-6
STATUS=OK

Campo	Descripción
ACTION	Acción recomendada por ARM64.
TARGET	Sensor, zona o actuador relacionado con la acción.
RISK	Nivel de riesgo: LOW, MEDIUM, HIGH o CRITICAL.
REASON	Motivo técnico de la decisión.
VALUE	Valor actual relevante.
INDICATOR	Indicador principal usado en la decisión.
STATUS	Resultado de la ejecución: OK o ERROR.

4.12 Manejo de errores

El motor ARM64 deberá responder con un mensaje de error estructurado cuando reciba datos incompletos, valores no numéricos, columnas faltantes o una lectura inválida.

Formato sugerido:

STATUS=ERROR
ERROR=INVALID_INPUT
DETAIL=EXPECTED_7_FIELDS

Python deberá registrar estos errores en MongoDB y evitar la ejecución de acciones físicas si la respuesta de ARM64 no es válida.

4.13 Componente B: Analizador histórico ARM64

El analizador histórico ARM64 será un conjunto de módulos encargados de procesar archivos de lecturas generados por el sistema del invernadero. Estos archivos deberán contener datos históricos con cantidad variable de filas.

El objetivo del analizador no será únicamente leer un archivo, sino procesar una sección específica de datos seleccionada por el usuario, interpretar una columna como una serie temporal y calcular indicadores que permitan analizar el comportamiento histórico del sensor.

4.14 Entrada del analizador

El programa ARM64 deberá recibir como parámetros mínimos:

archivo_entrada linea_inicial linea_final columna_sensor

Ejemplo:

lecturas.csv 10 80 TEMP

En este caso, el analizador deberá procesar únicamente las lecturas comprendidas entre la línea 10 y la línea 80 del archivo lecturas.csv, utilizando la columna TEMP como variable de análisis.

4.15 Columnas válidas

Las columnas válidas para análisis serán:

Columna	Descripción
TEMP	Temperatura ambiental.
HUM_AIRE	Humedad ambiental.
SOIL1	Humedad del suelo del área 1.
SOIL2	Humedad del suelo del área 2.

LUZ	Nivel de iluminación.
GAS	Nivel de gas o humo.

Cada columna seleccionada será interpretada como el eje Y de una serie temporal. El eje X será el orden de las lecturas dentro del rango seleccionado.

4.16 Validaciones obligatorias

Antes de ejecutar cualquier cálculo, el analizador histórico deberá validar:

1. Que el archivo exista.
2. Que el archivo tenga encabezado o formato documentado.
3. Que la línea inicial sea mayor o igual a 1.
4. Que la línea final sea mayor o igual que la línea inicial.
5. Que ambas líneas existan dentro del archivo.
6. Que la columna seleccionada exista.
7. Que los valores de la columna seleccionada sean numéricos.
8. Que el rango contenga suficientes datos para el cálculo solicitado.
9. Que el resultado pueda generarse sin división entre cero.
10. Que el resultado pueda generarse sin desbordamientos evidentes.

Si alguna validación falla, el programa deberá devolver un error estructurado y no deberá generar resultados incorrectos.

Ejemplo:

```
MODULE=HISTORICAL_ANALYZER  
STATUS=ERROR  
ERROR=INVALID_RANGE  
DETAIL=END_LINE_EXCEEDS_FILE_LENGTH
```

4.17 Actualización de módulos ARM64 existentes

Los módulos ARM64 desarrollados en la Fase 1 deberán mantenerse y reutilizarse en la Fase 2, pero con una modificación importante: ya no trabajarán sobre una cantidad fija de datos.

En esta fase, los algoritmos deberán adaptarse para procesar una cantidad variable de lecturas determinada por el rango seleccionado por el usuario.

Esto implica que cada módulo deberá ser capaz de:

1. Leer únicamente las filas comprendidas entre la línea inicial y la línea final indicadas.
2. Determinar dinámicamente cuántos datos contiene el rango seleccionado.
3. Ajustar sus ciclos y acumuladores al número real de muestras disponibles.
4. Generar resultados correctos independientemente de si el rango contiene pocas lecturas o muchas.

5. Validar que exista una cantidad mínima de datos cuando el cálculo lo requiera.

Los cálculos implementados en la Fase 1 deberán conservar su funcionalidad original, pero ahora deberán operar sobre ventanas de tamaño variable.

4.18 Salida del analizador

El resultado del análisis deberá generarse en formato estructurado. Cada módulo ARM64 deberá devolver información suficiente para identificar:

- cálculo ejecutado;
- columna analizada;
- rango procesado;
- cantidad de datos utilizados;
- resultados obtenidos;
- estado de ejecución.

4.19 Nuevos cálculos ARM64 de Fase 2

Además de actualizar los cálculos ARM64 existentes, se deberán implementar nuevos cálculos históricos para analizar el comportamiento de una variable dentro de un rango seleccionado.

Todos los cálculos deberán trabajar sobre una sola columna seleccionada y sobre todas las filas comprendidas entre la línea inicial y la línea final indicadas por el usuario.

Para todos los cálculos:

- Y representa los valores de la columna seleccionada.
- Y_i representa un valor específico de la serie.
- X representa la posición temporal implícita de cada lectura.
- N representa la cantidad de datos procesados.
- Si la ventana tiene N datos, entonces $X = 0, 1, 2, \dots, N - 1$.
- Todas las operaciones deberán realizarse en ARM64.
- Python solo deberá enviar parámetros, ejecutar los módulos y leer resultados.
- Todas las divisiones deberán ser enteras y truncadas.

4.19.1 Rutina 1: RMSE respecto a un valor ideal

Esta rutina deberá calcular qué tanto se aleja la variable analizada respecto a un valor ideal.

Archivo sugerido: modulo_1_rmse.s

Salida sugerida: resultado_rmse.txt

Fórmulas:

$$\text{ERROR}_i = Y_i - \text{IDEAL}$$

$ERROR2_i = ERROR_i * ERROR_i$

$MSE = \text{suma}(ERROR2_i) / N$

$RMSE = \text{sqrt_entera}(MSE)$

Donde:

- IDEAL representa el valor esperado para la variable.
- sqrt_entera representa una raíz cuadrada entera truncada.

Salida:

CALC=RMSE
COLUMN=SOIL1
WINDOW_START=1000
WINDOW_END=1050
COUNT=51
IDEAL=55
RMSE=18
STATUS=OK

4.19.2 Rutina 2: Regresión lineal simple

Esta rutina deberá calcular la tendencia general de la variable dentro de la ventana seleccionada.

No se aceptará calcular la tendencia únicamente con el primer y último valor.

Archivo sugerido: modulo_2_regresion.s

Salida sugerida: resultado_regresion.txt

Fórmula:

$NUMERADOR = (N * \text{suma}(X_i * Y_i)) - (\text{suma}(X_i) * \text{suma}(Y_i))$

$DENOMINADOR = (N * \text{suma}(X_i * X_i)) - (\text{suma}(X_i) * \text{suma}(X_i))$

$M_X100 = (NUMERADOR * 100) / DENOMINADOR$

Donde:

- M_X100 representa la pendiente multiplicada por 100.
- El resultado deberá ser entero truncado.

Clasificación:

- Si $M_X100 > 0$, la tendencia es ascendente.
- Si $M_X100 < 0$, la tendencia es descendente.
- Si $M_X100 = 0$, la tendencia es estable.

Salida:

```
CALC=LINEAR_REGRESSION  
COLUMN=TEMP  
WINDOW_START=1000  
WINDOW_END=1050  
COUNT=51  
SLOPE_X100=-42  
TREND=DESCENDING  
STATUS=OK
```

4.19.3 Rutina 3: Predicción futura por regresión

Esta rutina deberá estimar un valor futuro usando la recta calculada mediante regresión lineal.

La predicción se realizará para K lecturas futuras. Si no se recibe K, podrá usarse K = 5.

Archivo sugerido: modulo_3_prediccion.s

Salida sugerida: resultado_prediccion.txt

Fórmulas:

$$B_X100 = ((\text{suma}(Y_i) * 100) - (M_X100 * \text{suma}(X_i))) / N$$

$$X_FUTURE = N + K$$

$$Y_PRED = ((M_X100 * X_FUTURE) + B_X100) / 100$$

Donde:

- B_X100 representa el intercepto multiplicado por 100.
- X_FUTURE representa la posición futura estimada.
- Y_PRED representa el valor futuro estimado.

Salida sugerida:

```
CALC=PREDICTION  
COLUMN=SOIL1  
WINDOW_START=1000  
WINDOW_END=1050  
COUNT=51  
K=5  
SLOPE_X100=-42  
INTERCEPT_X100=6920  
PREDICTED_5=27  
STATUS=OK
```

4.19.4 Rutina 4: Integral del error por regla del trapecio

Esta rutina deberá calcular el error acumulado de la variable respecto a un valor ideal.

Archivo sugerido: modulo_4_integral_error.s

Salida sugerida: resultado_integral.txt

Fórmulas:

$ERROR_i = \text{abs}(Y_i - IDEAL)$

$ERROR_NEXT = \text{abs}(Y_{(i+1)} - IDEAL)$

$AREA_TRAPECIO = (ERROR_i + ERROR_NEXT) / 2$

$AREA_ERROR = AREA_ERROR + AREA_TRAPECIO$

El cálculo deberá repetirse desde $i = 0$ hasta $i = N - 2$.

Donde:

- $AREA_TRAPECIO$ representa el área aproximada entre dos errores consecutivos.
- $AREA_ERROR$ representa la suma acumulada de todas las áreas.

Salida sugerida:

```
CALC=ERROR_INTEGRAL
COLUMN=SOIL1
WINDOW_START=1000
WINDOW_END=1050
COUNT=51
IDEAL=55
ERROR_INTEGRAL=740
STATUS=OK
```

4.19.5 Rutina 5: Derivada suavizada por regresión local

Esta rutina deberá calcular la velocidad de cambio local usando mini-ventanas internas de 5 puntos.

No se aceptará calcular la derivada únicamente como resta entre dos puntos consecutivos.

Archivo sugerido: modulo_5_derivada_local.s

Salida sugerida: resultado_derivada.txt

Para cada mini-ventana:

$X = 0, 1, 2, 3, 4$

Fórmula:

$LOCAL_NUMERADOR = (5 * \text{suma}(X_i * Y_i)) - (\text{suma}(X_i) * \text{suma}(Y_i))$

$LOCAL_DENOMINADOR = (5 * \text{suma}(X_i * X_i)) - (\text{suma}(X_i) * \text{suma}(X_i))$

$LOCAL_SLOPE_X100 = (LOCAL_NUMERADOR * 100) / LOCAL_DENOMINADOR$

Como para $X = 0, 1, 2, 3, 4$:

$\text{suma}(X_i) = 10$

$\text{suma}(X_i * X_i) = 30$

$LOCAL_DENOMINADOR = 50$

Entonces:

$LOCAL_SLOPE_X100 = ((5 * \text{suma}(X_i * Y_i) - 10 * \text{suma}(Y_i)) * 100) / 50$

El programa deberá calcular todas las pendientes locales posibles y reportar el mayor cambio local detectado.

Salida:

CALC=LOCAL_DERIVATIVE
COLUMN=GAS
WINDOW_START=1000
WINDOW_END=1050
COUNT=51
WINDOW_SIZE=5
MAX_LOCAL_SLOPE_X100=420
STATUS=OK

El analizador histórico deberá imprimir una salida estructurada como la siguiente:

WINDOW_START=1000
WINDOW_END=1050
COLUMN=SOIL1
COUNT=51
IDEAL=55
RMSE=18
SLOPE_X100=-42
TREND=DESCENDING
PREDICTED_10=27
ERROR_INTEGRAL=740
MAX_LOCAL_SLOPE_X100=420
MAX_ACCELERATION_X100=180
RECOMMENDATION=RIEGO_PREVENTIVO_AREA_1

REASON=RMSE_ALTO_PREDICCION_BAJA_TENDENCIA_CONFIABLE
STATUS=OK

Restricciones obligatorias

Los módulos ARM64 deberán cumplir con las siguientes restricciones:

- Los cálculos principales deberán implementarse en ARM64.
- Python no deberá calcular RMSE, regresión, predicción, integral del error ni derivada local.
- Python únicamente podrá ejecutar los módulos, enviar parámetros y leer resultados.
- No se permite el uso de punto flotante.
- Todas las divisiones deberán ser enteras y truncadas.
- RMSE, regresión, predicción e integral requieren al menos 2 datos.
- La derivada suavizada por regresión local requiere al menos 5 datos.
- Si ocurre un error, el módulo deberá devolver una salida estructurada.

Salida de error:

STATUS=ERROR
ERROR=INSUFFICIENT_DATA
DETAIL=LOCAL_DERIVATIVE_REQUIRES_AT_LEAST_5_VALUES

4.20 Integración entre Python y ARM64

Python será el coordinador general del sistema, pero no deberá calcular la decisión automática principal.

Responsabilidades de Python

Responsabilidad	Descripción
Lectura de sensores	Leer sensores reales desde Raspberry Pi.
Comunicación con ARM64	Enviar lecturas al motor ARM64 en vivo y leer respuestas.
Ejecución de módulos históricos	Ejecutar los módulos ARM64 históricos cuando el usuario solicite análisis.
Interpretación de respuestas	Leer respuestas estructuradas generadas por ARM64.
Validación de acciones	Confirmar que las acciones recibidas sean acciones permitidas.
Control de GPIO	Ejecutar actuadores físicos según la decisión recibida.
MQTT	Publicar o recibir datos mediante MQTT.

MongoDB	Registrar lecturas, decisiones, resultados y errores.
Dashboard	Enviar datos al dashboard web.
Grafana	Permitir visualización de lecturas y resultados.

Responsabilidades de ARM64

Responsabilidad	Descripción
Entrada de datos	Recibir datos enviados por Python o parámetros del analizador histórico.
Validación	Validar formato básico de entrada.
Conversión	Parsear valores numéricos.
Historial	Mantener lecturas recientes en arreglos.
Indicadores	Calcular promedio, tendencia, amplitud u otros indicadores.
Decisión	Generar la decisión automática principal.
Análisis histórico	Procesar rangos y columnas seleccionadas.
Resultados	Generar salidas estructuradas.

4.21 Registro de resultados en MongoDB

Los resultados generados por ARM64 deberán almacenarse en MongoDB para permitir trazabilidad y visualización posterior.

Se recomienda crear o actualizar una colección llamada:

arm64_results

Cada documento almacenado deberá incluir como mínimo:

Campo	Descripción
timestamp	Fecha y hora de ejecución.
source	Origen del resultado: live_engine o historical_analyzer.
module	Nombre del módulo ARM64 ejecutado.
input	Datos de entrada utilizados.

range	Línea inicial y línea final, cuando aplique.
column	Columna analizada, cuando aplique.
result	Valores calculados por ARM64.
decision	Acción recomendada o ejecutada, cuando aplique.
risk	risk Nivel de riesgo calculado, cuando aplique.
status	OK o ERROR.
error_detail	Descripción del error, si existe.

4.22 Visualización en Grafana y dashboard web

Grafana deberá utilizarse para visualizar datos históricos y resultados generados por ARM64. No será suficiente mostrar únicamente lecturas crudas de sensores. Los paneles deberán permitir observar el comportamiento del invernadero y la relación entre lecturas, indicadores ARM64 y acciones ejecutadas.

Paneles mínimos sugeridos en Grafana

1. Lecturas históricas de temperatura.
2. Lecturas históricas de humedad ambiental.
3. Lecturas históricas de humedad de suelo por área.
4. Lecturas históricas de luz.
5. Lecturas históricas de gas.
6. Decisiones emitidas por el motor ARM64 en vivo.
7. Acciones ejecutadas por Python.
8. Indicadores calculados por ARM64.
9. Resultados del analizador histórico por rango y columna.
10. Errores o ejecuciones fallidas de módulos ARM64.
11. Comparación entre lectura real y predicción generada por ARM64.
12. Nivel de riesgo del invernadero a lo largo del tiempo.

Grafana será utilizado únicamente para visualización. No deberá controlar actuadores físicos.

Dashboard web

El dashboard web deberá permitir:

- consultar el estado actual del sistema;
- visualizar lecturas recientes;
- visualizar decisiones recientes generadas por ARM64;
- mostrar resultados ARM64;

- solicitar análisis histórico indicando archivo, línea inicial, línea final y columna;
- mostrar errores estructurados;
- permitir control remoto autorizado;
- proteger el acceso mediante login.

4.23 Login y despliegue público

La aplicación web deberá estar desplegada en un servicio accesible públicamente, como Render, Vercel, Amazon S3 o plataforma equivalente autorizada.

El dashboard principal no deberá ser accesible sin autenticación.

El sistema de login deberá cumplir como mínimo con:

1. Pantalla de inicio de sesión.
2. Validación de usuario y contraseña.
3. Cierre de sesión.
4. Usuario de prueba documentado para evaluación.
5. Restricción de acceso al dashboard si no existe sesión activa.

La documentación deberá incluir:

1. URL pública del dashboard.
2. Usuario de prueba.
3. Contraseña de prueba.
4. Instrucciones para acceder.
5. Capturas o evidencia del dashboard protegido.

4.25 Recursos y herramientas a utilizar

Recurso	Uso dentro del proyecto
Raspberry Pi	Unidad principal de control del invernadero.
Python	Coordinación del sistema, sensores, GPIO, MongoDB, MQTT y ejecución de ARM64.
ARM64 Assembly	Implementación del motor de decisión y análisis histórico.
MongoDB Atlas	Persistencia de lecturas, decisiones, eventos, resultados y errores.
MQTT	Comunicación IoT de lecturas y comandos.
Grafana	Visualización avanzada de históricos, indicadores y resultados ARM64.
Dashboard web	Monitoreo, control remoto y consulta de resultados.
Servicio de despliegue	Publicación web mediante Render, Vercel, Amazon S3 u otro servicio equivalente.

GDB	Depuración y defensa de módulos ARM64.
Makefile	Compilación y ejecución ordenada de módulos ARM64.
GitHub	Control de versiones y entrega del código fuente.

4.25 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código fuente	Repositorio completo con Fase 1 funcional y Fase 2 integrada.
Código ARM64	Módulos ARM64 nuevos, uno por integrante, separados y documentados.
Motor en vivo	Programa ARM64 ejecutado desde Python por stdin/stdout.
Analizador histórico	Programa ARM64 capaz de leer archivos grandes, validar rango inicio-fin y analizar una columna.
Python Bridge	Código Python encargado de enviar datos a ARM64, leer respuestas y ejecutar acciones.
MongoDB	Colecciones actualizadas con lecturas, eventos, decisiones ARM64 y resultados históricos.
Grafana	Dashboard de Grafana con paneles de lecturas, decisiones y cálculos ARM64.
Dashboard web	Aplicación web desplegada públicamente con login.
URL pública	Enlace funcional de Render, Vercel, Amazon S3 o equivalente autorizado.
Credenciales de prueba	Usuario y contraseña de prueba para acceder al dashboard.
Documentación técnica	Explicación de arquitectura, flujo Python-ARM64, módulos ARM64, cálculos, despliegue y uso.
Evidencia GDB	Capturas o explicación de depuración de cada módulo ARM64.
Defensa individual	Cada integrante deberá explicar su módulo ARM64 y su integración.

5. Material de apoyo

Categoría	Link	Descripción
Documentación oficial	https://docs.python.org/3/library/subprocess.html	Uso de subprocess para comunicación entre Python y procesos externos.
Documentación oficial	https://grafana.com/docs/	Documentación de Grafana para dashboards y visualización.
Documentación oficial	https://www.mongodb.com/docs/	Documentación de MongoDB Atlas.
Documentación oficial	https://vercel.com/docs	Despliegue de aplicaciones web en Vercel.
Documentación oficial	https://render.com/docs	Despliegue de aplicaciones en Render.
Documentación oficial	https://docs.aws.amazon.com/AWS/latest/userguide/WebsiteHosting.html	Hospedaje estático en Amazon S3.
Referencia técnica	Material del curso	Guías de ARM64, syscalls, manejo de archivos, registros, GDB y estructuras en memoria.

6. Metodología

Pasos a seguir

1. Verificar Fase 1.

Confirmar que el invernadero físico, sensores, actuadores, MQTT, MongoDB, dashboard, LCD y botones físicos funcionan correctamente.

2. Definir flujo de Fase 2.

Establecer cómo Python enviará lecturas al motor ARM64 y cómo interpretará las respuestas para ejecutar acciones.

3. Diseñar formato de comunicación Python-ARM64.

Definir el formato de entrada por stdin y salida por stdout.

4. Implementar utils.s.

Crear funciones comunes para conversión, lectura, escritura, recorridos de arreglos y validaciones básicas.

5. Implementar almacenamiento de historial.

Crear arreglos de tamaño fijo para almacenar lecturas recientes por sensor.

6. Implementar indicadores.

Desarrollar promedio reciente, tendencia acumulada y amplitud reciente.

7. Implementar motor ARM64 en vivo.

Crear el programa ARM64 que recibe lecturas, actualiza historial, calcula indicadores y genera decisiones.

8. Integrar con Python.

Python debe ejecutar ARM64, enviar datos, recibir respuestas y ejecutar actuadores.

9. Implementar analizador histórico.

Crear el programa ARM64 que lee archivos, valida rangos y extrae una columna dentro de una ventana de líneas.

10. Actualizar cálculos existentes.

Adaptar los cálculos anteriores para trabajar con cantidad variable de datos.

11. Implementar nuevos cálculos ARM64.

12. Registrar resultados.

Guardar lecturas, decisiones, cálculos históricos, recomendaciones y acciones ejecutadas en MongoDB.

13. Configurar Grafana.

Crear paneles para visualizar lecturas, decisiones ARM64 y resultados de análisis.

14. Desplegar aplicación web.

Publicar la aplicación en Render, Vercel, Amazon S3 o plataforma equivalente autorizada.

15. Agregar login.

Proteger el dashboard mediante autenticación.

16. Documentar.

Explicar arquitectura, módulos, fórmulas, flujo de datos, despliegue y pruebas.

17. Preparar defensa.

Cada integrante debe preparar la explicación técnica de su módulo ARM64.

7. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

7.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple Sí/No
Fase 1 funcional	El grupo demuestra que la Fase 1 sigue funcionando: maqueta física, sensores, actuadores, MQTT, MongoDB, dashboard, LCD, botones físicos y registro de datos.	
Maqueta física	Presenta un invernadero físico funcional con dos áreas de cultivo y centro de control. No se acepta únicamente protoboard.	
Raspberry Pi funcional	El sistema corre sobre Raspberry Pi 3 o 4 y mantiene integración con sensores y actuadores.	
Sensores obligatorios	Integra sensores ambientales, sensores de humedad de suelo, sensor de luz y sensor de gas.	
Actuadores obligatorios	Funcionan riego, ventilación, luces, buzzer y LEDs de estado.	
Python GPIO	Python controla lectura de sensores, ejecución de actuadores y comunicación con ARM64.	
MQTT	Se publican lecturas y se reciben comandos mediante MQTT.	

MongoDB Atlas	Se almacenan lecturas, eventos, comandos, estados, decisiones ARM64 y resultados históricos.	
Dashboard web	Existe dashboard web funcional para monitoreo, consulta de resultados y control remoto autorizado.	
Login	El dashboard web cuenta con autenticación. No se permite acceso directo al panel principal sin iniciar sesión.	
Despliegue público	La aplicación web está disponible mediante URL pública usando Render, Vercel, Amazon S3 o plataforma equivalente autorizada.	
Grafana	Existe integración con Grafana para visualizar lecturas, decisiones ARM64, indicadores y resultados del análisis histórico.	
Motor ARM64 en vivo	Existe un programa ARM64 ejecutado desde Python que recibe datos por stdin, mantiene historial reciente y responde acciones por stdout.	
Decisión en ARM64	La decisión automática principal es generada por ARM64. Python solo interpreta y ejecuta la acción indicada.	
Analizador histórico ARM64	Existe un programa ARM64 que procesa archivos de lecturas, recibe línea inicial, línea final y columna seleccionada.	
Validación de rango	El analizador valida que la línea inicial y final existan dentro del archivo y que el rango sea válido.	
Selección de columna	El analizador permite seleccionar una columna de sensor y procesarla como serie temporal.	
Actualización de módulos existentes	Los módulos ARM64 desarrollados previamente se adaptan para trabajar con cantidad variable de datos.	
Cálculos ARM64 nuevos	Se implementan los cálculos nuevos de Fase 2: RMSE respecto a un valor ideal, regresión lineal simple, predicción futura por regresión, integral del error por regla del trapecio y derivada suavizada por regresión local.	
Módulo ARM64 individual	Cada integrante entrega, explica y defiende un módulo ARM64 propio.	

Evidencia GDB	Cada integrante presenta evidencia o explicación de depuración con GDB de su módulo ARM64.	
Repositorio accesible	El auxiliar o docente tiene acceso al repositorio completo con código actualizado e instrucciones de ejecución.	
Organización del proyecto	El repositorio separa de forma clara el contenido de Fase 1 y Fase 2. No se impone una estructura única, pero debe ser comprensible y ejecutable.	
Documentación técnica	Se entrega documentación con arquitectura, flujo Python-ARM64, módulos ARM64, fórmulas, despliegue, pruebas y errores.	

7.2 Detalle de la Calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos indicados en esta sección. De no cumplir con los requisitos críticos, el proyecto podrá recibir una nota de cero puntos o no optar a clasificación completa.

Descripción de ponderación	Valor	Observación	Punteo
HABILIDADES	10	Organización, documentación y defensa técnica general.	0
Organización del repositorio	3	El proyecto debe estar ordenado y separar claramente los componentes de Fase 1 y Fase 2. La estructura interna queda a criterio del grupo, siempre que sea comprensible y ejecutable.	
Documentación de Fase 2	3	Debe explicar arquitectura, flujo Python-ARM64, módulos ARM64, fórmulas, pruebas, errores, despliegue y uso del sistema.	
Defensa técnica general	4	El grupo debe explicar el funcionamiento general del sistema, la integración entre hardware, Python, ARM64, MongoDB, dashboard, Grafana y actuadores.	
Subtotal Habilidades	10		0
CONOCIMIENTOS	90	Implementación técnica, integración y	0

		pruebas.	
Motor ARM64 en vivo	20	Programa ARM64 ejecutado desde Python para generar decisiones automáticas.	0
Comunicación Python-ARM64	4	Python debe enviar lecturas al programa ARM64 y recibir una respuesta válida mediante stdin/stdout o mecanismo equivalente documentado.	
Lectura y validación de datos	3	ARM64 debe recibir, separar, convertir y validar correctamente los valores enviados por Python.	
Historial de lecturas	4	Debe mantenerse una estructura con las últimas lecturas recibidas para apoyar la toma de decisiones.	
Indicadores calculados en ARM64	4	Deben calcularse indicadores obtenidos a partir del historial, como promedio reciente, tendencia acumulada o amplitud reciente.	
Decisión generada por ARM64	3	La acción principal debe ser determinada por ARM64 aplicando los indicadores y el orden de prioridad definido.	
Salida estructurada y registro	2	ARM64 debe devolver una respuesta clara y Python debe registrar la decisión, indicadores y resultados en MongoDB.	
Analizador histórico ARM64	20	Procesamiento de archivos, rango, columna y salida estructurada.	0
Lectura de archivos con tamaño variable	3	Debe funcionar con archivos que tengan distinta cantidad de registros.	
Selección de rango	3	Debe permitir indicar línea inicial y línea final para analizar una ventana específica de datos.	
Validación de rango	4	Debe verificar que las líneas solicitadas existan dentro del archivo y que el rango sea válido.	
Selección de columna	3	Debe permitir elegir qué variable analizar.	
Procesamiento de datos del rango	4	Debe procesar únicamente los datos comprendidos entre la línea inicial y la línea final seleccionadas.	
Resultados y errores	3	Debe generar resultados claros y reportar errores estructurados cuando el archivo,	

		rango, columna o datos sean inválidos.	
Módulos ARM64 individuales y cálculos	15	Módulos actualizados, nuevos cálculos y defensa individual.	0
Actualización de módulos existentes	3	Los módulos ARM64 desarrollados previamente deberán adaptarse para trabajar con cantidad variable de datos.	
Nuevos cálculos ARM64 de Fase 2	5	Se implementan los cálculos definidos en la sección 4.19: RMSE respecto a un valor ideal, regresión lineal simple, predicción futura por regresión, integral del error por regla del trapecio y derivada suavizada por regresión local.	
Módulo propio por integrante	2	Cada integrante debe desarrollar o actualizar un módulo ARM64 diferente y defendible.	
Funcionamiento correcto del módulo	2	El módulo debe producir resultados correctos según la fórmula implementada y el rango seleccionado.	
Uso de ARM64	2	El cálculo principal debe estar desarrollado en ensamblador ARM64, no en Python, C u otro lenguaje.	
Evidencia de ejecución o GDB	1	Debe demostrarse que el módulo compila, funciona y puede depurarse o explicarse mediante GDB.	
Integración Python, ARM64, MongoDB y actuadores	10	Coordinación del sistema y persistencia de resultados.	0
Python como coordinador	3	Python debe encargarse de leer sensores, ejecutar ARM64, interpretar respuestas y coordinar el flujo general del sistema.	
Acciones ejecutadas desde decisiones ARM64	3	Las acciones físicas del sistema deben responder a las decisiones generadas por ARM64.	
Registro en MongoDB	3	Deben almacenarse lecturas, decisiones, indicadores, resultados, errores, rangos y columnas asociadas a Fase 2.	
Manejo de errores de integración	1	Python debe evitar ejecutar acciones si ARM64 devuelve una respuesta inválida o un error.	
Grafana, dashboard, login y	15	Visualización, acceso y publicación del sistema.	0

despliegue			
Uso de Grafana	5	Debe mostrar información relacionada con lecturas, decisiones ARM64, indicadores y resultados del análisis histórico.	
Dashboard actualizado	3	Debe incluir información nueva generada por Fase 2, como decisiones ARM64, resultados históricos, indicadores y errores.	
Login funcional	3	Debe existir autenticación básica para acceder al dashboard y proteger rutas privadas.	
Despliegue público	3	La aplicación debe estar disponible mediante una URL pública accesible para evaluación.	
Datos reales en la visualización	1	La información mostrada debe provenir del sistema y no ser únicamente estática o simulada sin justificación.	
Pruebas, evidencia y defensa	10	Evidencia de funcionamiento, pruebas y explicación técnica.	0
Pruebas del motor ARM64	3	Debe demostrarse el funcionamiento del motor en vivo con entradas válidas e inválidas y sus respuestas.	
Pruebas del analizador histórico	3	Deben mostrarse casos válidos e inválidos del análisis histórico, incluyendo rangos y columnas.	
Evidencias de funcionamiento	2	Deben presentarse capturas, registros, resultados o video demostrativo del sistema integrado.	
Defensa individual	2	Cada integrante debe explicar su módulo ARM64, registros utilizados, ciclos, memoria, fórmula, validaciones e integración.	
Subtotal Conocimientos	90		0
TOTAL	100		0

7.3 Comentarios Generales

Situación	Penalización
-----------	--------------

No mantener funcional la Fase 1 o no contar con los elementos base necesarios para ejecutar Fase 2, como maqueta, Raspberry Pi, sensores, actuadores, MQTT, MongoDB o dashboard base	Penalización grupal: no se califica Fase 2 o hasta 100% del proyecto, según gravedad.
Presentar únicamente componentes conectados en protoboard, sin maqueta física de invernadero con dos áreas de cultivo y centro de control	Penalización grupal: no se acepta como entrega válida.
No presentar repositorio accesible, código fuente completo, instrucciones de ejecución o archivos necesarios para compilar y probar Fase 2	Penalización grupal: no se califica el proyecto o hasta 30% del proyecto, según gravedad.
Python realiza la decisión automática principal, calcula los indicadores principales o sustituye el trabajo que debe realizar ARM64	Penalización grupal o individual: hasta 30% del proyecto o pérdida total del criterio correspondiente.
No implementar o no integrar correctamente el motor ARM64 en vivo con Python mediante stdin/stdout o mecanismo equivalente documentado	Penalización grupal: hasta 25% del proyecto.
El motor ARM64 en vivo no mantiene historial de lecturas, no calcula indicadores recientes, no justifica decisiones o decide únicamente con umbrales simples	Penalización grupal: hasta 15% del proyecto.
No implementar el analizador histórico ARM64 o implementarlo sin integración con Python, MongoDB, dashboard o Grafana	Penalización grupal: hasta 20% del proyecto.
El analizador histórico no procesa archivos con cantidad variable de filas, usa una cantidad fija de registros, tiene ciclos quemados o no ajusta sus cálculos al rango seleccionado	Penalización grupal o individual: hasta 15% del proyecto o pérdida del criterio correspondiente.
No permitir seleccionar línea inicial, línea final o columna de sensor, o no interpretar la columna como eje Y y el orden de lectura como eje X temporal	Penalización grupal: hasta 10% del proyecto.
No validar rangos, archivos, columnas o datos inválidos; aceptar líneas inexistentes, rangos invertidos, valores no numéricos o casos que producen división entre cero	Penalización grupal o individual: hasta 10% del proyecto.
No implementar los cálculos ARM64 obligatorios o realizarlos en Python, C u otro lenguaje en lugar de ARM64	Penalización grupal o individual: no se califica el cálculo correspondiente.

No presentar al menos un módulo ARM64 propio por integrante, presentar módulos copiados entre integrantes o sin diferencias sustanciales	Penalización individual: hasta 100% del componente ARM64 de los involucrados.
Un integrante no puede explicar su módulo ARM64, registros, ciclos, memoria, fórmula, validaciones o integración con Python	Penalización individual: hasta 100% del componente individual.
Documentación técnica incompleta, inconsistente con el código o sin explicación de arquitectura, módulos ARM64, fórmulas, pruebas, errores y ejecución	Penalización grupal: hasta 10% del proyecto.
No asistir, no participar o no defender técnicamente el trabajo realizado	Penalización individual: el estudiante no opta a la nota completa.
Entrega fuera de tiempo, archivos dañados, dependencias no documentadas o imposibilidad de ejecutar el proyecto por falta de instrucciones	Penalización grupal o individual: según normativa del curso o hasta el total del componente afectado.

- No se aceptará que Python realice los cálculos principales asignados a ARM64.
- No se aceptará que un integrante sea evaluado únicamente por maqueta, Python, dashboard, Grafana o documentación.
- Cada integrante deberá tocar código ARM64 y defenderlo individualmente.
- El archivo histórico puede tener cantidad variable de filas, pero el sistema deberá validar rangos inválidos.
- El análisis histórico se realizará sobre una sola columna seleccionada.
- El eje X se tomará como el orden temporal implícito de las lecturas dentro de la ventana.
- El eje Y será el valor del sensor seleccionado.
- Grafana se usará para visualizar datos, no para sustituir los cálculos ARM64.
- El dashboard web debe estar disponible públicamente y protegido con login.
- La defensa podrá incluir pruebas manuales de ARM64, revisión de código, ejecución con archivos de prueba y explicación con GDB.
- TOMAR EN CUENTA QUE PUEDE OCURRIR CAMBIOS CON TEMA DE LA CALIFICACIÓN SI EL INGENIERO ASÍ LO DECIDE.